

PHP Introdução

Estruturas de Repetição • Escopo • Funções

Cronograma

- Bloco 1 — Loop for: fundamentos e padrões
- Bloco 2 — while e do...while: quando usar cada um
- Bloco 3 — Escopo: global, function, superglobals, static
- Bloco 4 — Funções: parâmetros, type hints, return, defaults
- Bloco 5 — Projeto Integrador: "Calculadora de Estatísticas"

Bloco 1: Loop for — Conceito & Padrões PHP

- Sintaxe: `for (inicialização; condição; incremento) { bloco }`
- Executa enquanto condição for true
- Ideal quando número de iterações é conhecido
- PHP também tem `foreach` para arrays (mais usado)

Bloco 1: Padrões Práticos de for (PHP)

```
<?php
// 1. Contagem simples 1 a 10
for ($i = 1; $i <= 10; $i++) {
    echo $i . PHP_EOL;
}

// 2. Contagem regressiva 10 a 1
for ($i = 10; $i >= 1; $i--) {
    echo $i . PHP_EOL;
}

// 3. Apenas pares de 1 a 20
for ($i = 2; $i <= 20; $i += 2) {
    echo $i . PHP_EOL;
}
```

```
// 4. Iterar array com índice
$frutas = ["maçã", "banana", "laranja", "uva"];
for ($i = 0; $i < count($frutas); $i++) {
    echo $i . " - " . $frutas[$i] . PHP_EOL;
}

// 5. Soma acumulada 1 a 100
$soma = 0;
for ($i = 1; $i <= 100; $i++) {
    $soma += $i;
}
echo $soma; // 5050

// 6. FOREACH (mais idiomático PHP)
foreach ($frutas as $indice => $fruta) {
    echo "$indice - $fruta" . PHP_EOL;
}
```

Bloco 1: Exercício 1.1 — Contagem Básica

Exercício 1.1 — Imprima números de 1 a 10, um por linha

Código inicial:

```
<?php
// Complete o loop for
for ($i = ____; $i <= ____; $i++) {
    echo ____ . PHP_EOL;
}
```

Bloco 1: Exercício 1.2 — Pares e Ímpares

Exercício 1.2 — Imprima apenas pares de 1 a 20, depois modifique para ímpares

Código inicial:

```
<?php
// PARES: complete o incremento
for ($i = 2; $i <= 20; $i += __) {
    echo $i . PHP_EOL;
}

// ÍMPARES: ajuste início e incremento
for ($i = __; $i <= 20; $i += __) {
    echo $i . PHP_EOL;
}
```

Bloco 1: Exercício 1.3 — Soma Acumulada

Exercício 1.3 — Some todos os números de 1 a 100 usando loop for

Código inicial:

```
<?php
$soma = 0;
// Complete o loop
for ($i = __; $i <= __; $i++) {
    $soma ____;
}
echo $soma; // Deve ser 5050
```

Bloco 1: Exercício 1.4 — Tabuada Interativa

Exercício 1.4 — Peça um número e imprima a tabuada de 1 a 10

Código inicial:

```
<?php
$n = (int)readline("Digite um número: ");
// Complete o loop para tabuada 1-10
for ($i = __; $i <= __; $i++) {
    echo "$n x $i = " . _____ . PHP_EOL;
}
```


Bloco 1: Exercício 1.5 — Iterar Array com Índice

Exercício 1.5 — Imprima cada fruta com seu índice (0-based)

Código inicial:

```
<?php
$frutas = ["maçã", "banana", "laranja", "uva",
"manga"];
// Complete: use count() na condição
for ($i = 0; $i < _____; $i++) {
    echo _____ . " - " . _____ . PHP_EOL;
}
```

Solução:

```
<?php
$frutas = ["maçã", "banana", "laranja", "uva",
"manga"];
for ($i = 0; $i < count($frutas); $i++) {
    echo $i . " - " . $frutas[$i] . PHP_EOL;
}
// Saída:
// 0 - maçã
// 1 - banana
// 2 - laranja
// 3 - uva
// 4 - manga
```

Bloco 1: Desafios — Pratique Mais

- 1.A — Fatorial: calcule $n!$ ($5! = 120$, $0! = 1$)
- 1.B — Fibonacci: primeiros 15 números (0,1,1,2,3,5,8...)
- 1.C — Triângulo de asteriscos altura 5
- Dica: use `var_dump()` ou `echo` para testar cada passo

Bloco 1: Desafio 1.A — Fatorial (Solução PHP)

```
<?php
function fatorial($n) {
    if ($n === 0) return 1;
    $resultado = 1;
    for ($i = 2; $i <= $n; $i++) {
        $resultado *= $i;
    }
    return $resultado;
}

echo fatorial(5) . PHP_EOL; // 120
echo fatorial(0) . PHP_EOL; // 1
echo fatorial(7) . PHP_EOL; // 5040
```

Bloco 1: Desafio 1.B — Fibonacci Iterativo (Solução PHP)

```
<?php
function fibonacci($qtd) {
    $seq = [0, 1];
    for ($i = 2; $i < $qtd; $i++) {
        $seq[] = $seq[$i-1] + $seq[$i-2];
    }
    return $seq;
}

print_r(fibonacci(15));
// Array
// (
//     [0] => 0
//     [1] => 1
//     [2] => 1
//     [3] => 2
//     [4] => 3
//     [5] => 5
//     [6] => 8
//     [7] => 13
//     [8] => 21
//     [9] => 34
//     [10] => 55
//     [11] => 89
//     [12] => 144
//     [13] => 233
//     [14] => 377
// )
```

Bloco 1: Desafio 1.C — Triângulo (Solução PHP)

```
<?php
$altura = 5;
for ($i = 1; $i <= $altura; $i++) {
    echo str_repeat("*", $i) . PHP_EOL;
}
// *
// **
// ***
// ****
// *****
```

Bloco 2: while e do...while — Quando Usar Cada Um

- for → iterações conhecidas/contáveis
- while → condição complexa, pode não executar nenhuma vez
- do...while → menus, validação, GARANTE pelo menos 1 execução
- Sintaxe idêntica ao JS, apenas \$ nas variáveis

Bloco 2: Sintaxe Comparada (PHP)

```
<?php
// WHILE – testa ANTES
$i = 1;
while ($i <= 5) {
    echo $i . PHP_EOL;
    $i++;
}

// DO...WHILE – executa 1x, testa DEPOIS
$j = 1;
do {
    echo $j . PHP_EOL;
    $j++;
} while ($j <= 5);

// Ambos imprimem 1,2,3,4,5
// Diferença: se condição for falsa initially
// while: 0 execuções | do...while: 1 execução
```

Bloco 2: Exercício 2.1 — Contagem com while

Exercício 2.1 — Refaça exercício 1.1 (1 a 10) usando while

Código inicial:

```
<?php
$i = 1;
// Complete a condição e o incremento
while (_____) {
    echo $i . PHP_EOL;
    _____;
}
```


Bloco 2: Exercício 2.2 — Soma Até Limite

Exercício 2.2 — Some números aleatórios (1-10) até soma > 100. Conte quantos números.

Código inicial:

```
<?php
$soma = 0;
$count = 0;
// Complete: enquanto soma <= 100
while (_____) {
    $num = rand(1, 10);
    $soma += $num;
    $count++;
}
echo "Soma: $soma, Números: $count" . PHP_EOL;
```

Bloco 2: Exercício 2.3 — Validação de Senha (do...while)

Exercício 2.3 — Peça senha até acertar "secreto123". Conte tentativas.

Código inicial:

```
<?php
$tentativas = 0;
$senha = "";
// Complete: do...while
_____ {
    $senha = readline("Digite a senha: ");
    $tentativas++;
} _____ ($senha !== "secreto123");

echo "Acertou em $tentativas tentativas" . PHP_EOL;
```

Bloco 2: Exercício 2.4 — Menu Interativo

Exercício 2.4 — Menu que repete até escolher "Sair": 1.Saudação 2.Data 3.Sair

Código inicial:

```
<?php
$opcao = "";
do {
    $opcao = readline("1.Saudação\n2.Data\n3.Sair\nEscolha:
");
    // Complete com switch
    switch ($opcao) {
        case "1":
            _____;
            break;
        case "2":
            _____;
            break;
        case "3":
            echo "Até logo!" . PHP_EOL;
            break;
        default:
            echo "Opção inválida" . PHP_EOL;
    }
} while (_____);
```

Bloco 2: Exercício 2.5 — Adivinhe o Número

Exercício 2.5 — Computador escolhe 1-100. Usuário adivinha com dicas. Conte tentativas.

Código inicial:

```
<?php
$segredo = rand(1, 100);
$chute = 0;
$tentativas = 0;
// Complete: enquanto chute != segredo
while (_____) {
    $chute = (int)readline("Chute (1-100):
");
    $tentativas++;
    if ($chute > $segredo) {
        _____;
    } elseif ($chute < $segredo) {
        _____;
    }
}
echo "Acertou em $tentativas!" . PHP_EOL;
```

Bloco 2: Desafios

- 2.A — Múltiplos de 3 ou 5 abaixo de 1000 (Project Euler #1)
- 2.B — Palavra Secreta: forme "PHP" digitando letras
- 2.C — Loop controlado: while(true) com break (negativo) / continue (zero)

Bloco 2: Desafio 2.A — Euler #1 (Solução PHP)

```
<?php
$soma = 0;
$i = 1;
while ($i < 1000) {
    if ($i % 3 === 0 || $i % 5 === 0) {
        $soma += $i;
    }
    $i++;
}
echo $soma . PHP_EOL; // 233168
```

Bloco 2: Desafio 2.C — Loop Infinito Controlado (Solução PHP)

```
<?php
$numeros = [];
while (true) {
    $entrada = (int)readline("Número (negativo=para, 0=pula): ");
    if ($entrada < 0) break;
    if ($entrada === 0) continue;
    $numeros[] = $entrada;
}
$media = array_sum($numeros) / count($numeros);
echo "Números: " . implode(", ", $numeros) . PHP_EOL;
echo "Média: $media" . PHP_EOL;
```

Bloco 3: Escopo em PHP — O Que Você Precisa Saber

- Global: variáveis fora de funções (use `$GLOBALS` ou `global $x`)
 - Local (Function): variáveis declaradas dentro de function
 - Static: mantém valor entre chamadas da função
-
- PHP NÃO tem escopo de bloco (if/for não criam escopo)
 - include/require: compartilham escopo do arquivo que incluiu

Bloco 3: Escopo na Prática PHP — Rode e Observe

```
<?php
// 1. ESCOPO GLOBAL vs LOCAL
$global = "sou global";
function teste() {
    $local = "sou local";
    global $global; // precisa declarar para
    modificar
    echo $global . PHP_EOL; // "sou global"
    echo $local . PHP_EOL; // "sou local"
}
teste();
// echo $local; // Notice: Undefined variable

// 2. STATIC – mantém estado entre chamadas
function contador() {
    static $count = 0;
    $count++;
    return $count;
}
```

```
echo contador() . PHP_EOL; // 1
echo contador() . PHP_EOL; // 2
echo contador() . PHP_EOL; // 3

// 3. SUPERGLOBALS – sempre acessíveis
// $_SERVER["PHP_SELF"], $_GET["id"],
// $_POST["nome"]
// $GLOBALS["variavel_global"]

// 4. SEM ESCOPO DE BLOCO
if (true) {
    $bloco = "vaza do if";
}
echo $bloco . PHP_EOL; // "vaza do if" –
FUNCIONA!
```

Bloco 3: Exercício 3.1 — Identifique os Escopos

Exercício 3.1 — Para cada echo, diga: funciona? imprime o quê? notice/error?

Código inicial:

```
<?php
$a = 1;
function teste() {
    $b = 2;
    if (true) {
        $c = 3;
        $d = 4; // sem var, vaza do if
        echo $a . PHP_EOL; // ?
        echo $b . PHP_EOL; // ?
        echo $c . PHP_EOL; // ?
        echo $d . PHP_EOL; // ?
    }
    echo $c . PHP_EOL; // ?
    echo $d . PHP_EOL; // ?
}
teste();
echo $d . PHP_EOL; // ?
```

Bloco 3: Exercício 3.2 — Global vs Local

Exercício 3.2 — Como acessar/modificar variável global dentro da function?

Código inicial:

```
<?php
$valor = "global";

function primeiro() {
    // Como ler $valor global aqui?
    echo _____ . PHP_EOL;

    // Como MODIFICAR $valor global aqui?
    _____;
    $valor = "modificado na function";
}

primeiro();
echo $valor . PHP_EOL; // Deve mostrar "modificado na function"
```

Bloco 3: Exercício 3.3 — Static: Contador Persistente

Exercício 3.3 — Crie função contador() que mantém valor entre chamadas usando static

Código inicial:

```
<?php
function contador() {
    // Declare $count como static com valor inicial 0
    _____ $count = 0;
    $count++;
    return $count;
}

echo contador() . PHP_EOL; // 1
echo contador() . PHP_EOL; // 2
echo contador() . PHP_EOL; // 3
```

Bloco 3: Exercício 3.4 — Superglobals

Exercício 3.4 — Acesse \$_SERVER, \$_GET, \$GLOBALS dentro de function sem global

Código inicial:

```
<?php
$config = "produção";

function mostrarAmbiente() {
    // Acesse $_SERVER["SERVER_NAME"] sem global
    echo "Servidor: " . _____ . PHP_EOL;

    // Acesse $config via $GLOBALS
    echo "Ambiente: " . _____ . PHP_EOL;
}

mostrarAmbiente();
```

Bloco 3: Exercício 3.5 — Include Compartilha Escopo

Exercício 3.5 — Arquivo incluído vê variáveis do arquivo que incluiu

Código inicial:

```
<?php
// arquivo: config.php
$apiKey = "secret-123";
$debug = true;

// arquivo: app.php
// include "config.php";
// Como acessar $apiKey e $debug aqui?

echo "API: $apiKey, Debug: " . ($debug ? "on" :
"off") . PHP_EOL;
```

Solução:

```
<?php
// config.php
$apiKey = "secret-123";
$debug = true;

// app.php
include "config.php";
// Variáveis do include ficam no MESMO
echo "API: $apiKey, Debug: " . ($debug
"off") . PHP_EOL;
// API: secret-123, Debug: on

// CUIDADO: include pode sobrescrever
variáveis!
```

Bloco 3: Desafios — Padrões Avançados

- 3.A — Singleton com static: apenas uma instância da classe
- 3.B — Registry Pattern: armazenar/configurar objetos globalmente

Bloco 3: Desafio 3.A — Singleton com Static (Solução)

```
<?php
class Database {
    private static $instance = null;
    private $connection;

    private function __construct() {
        $this->connection = new PDO("sqlite::memory:");
    }

    public static function getInstance() {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function getConnection() {
        return $this->connection;
    }
}

$db1 = Database::getInstance();
$db2 = Database::getInstance();
echo ($db1 === $db2) ? "Mesma instância!" : "Diferentes";
// Mesma instância!
```


Bloco 3: Desafio 3.B — Registry Pattern (Solução)

```
<?php
class Registry {
    private static $data = [];

    public static function set($key, $value) {
        self::$data[$key] = $value;
    }

    public static function get($key, $default = null) {
        return self::$data[$key] ?? $default;
    }

    public static function has($key) {
        return isset(self::$data[$key]);
    }
}

Registry::set("app.name", "MeuApp");
Registry::set("db.host", "localhost");

echo Registry::get("app.name") . PHP_EOL; // MeuApp
echo Registry::get("db.host") . PHP_EOL;  // localhost
echo Registry::get("nao.existe", "default") . PHP_EOL; // default
```

Bloco 4: Funções PHP — Parâmetros, Type Hints, Retorno

- Parâmetros = nomes na definição | Argumentos = valores na chamada
- Type hints (PHP 7+): `function soma(int $a, int $b): int { }`
- Defaults: `function f($a, $b = 10) { }` — obrigatórios primeiro
- Variadic: `function f(...$args) { }` — `$args` vira array
- Unpacking: `f(...[1,2,3])` — expande array em argumentos
- Return type declaration: `function f(): int { return 1; }`
- Strict types: `declare(strict_types=1);` no topo do arquivo

Bloco 4: Type Hints, Defaults, Variadic, Return Types

```
<?php
declare(strict_types=1);

// 1. TYPE HINTS + RETURN TYPE
function soma(int $a, int $b): int {
    return $a + $b;
}
echo soma(2, 3) . PHP_EOL; // 5
// soma("2", 3); // TypeError em strict_types

// 2. PARÂMETROS DEFAULT
function saudacao(string $nome, string $saudacao =
"Olá"): string {
    return "$saudacao, $nome!";
}
echo saudacao("Ana") . PHP_EOL;           // Olá,
Ana!
echo saudacao("Ana", "Oi") . PHP_EOL;    // Oi, Ana!
```

```
// 3. VARIADIC (...args)
function somaTudo(int ...$numeros): int {
    return array_sum($numeros);
}
echo somaTudo(1, 2, 3) . PHP_EOL;        // 6
echo somaTudo() . PHP_EOL;               // 0

// 4. UNPACKING (...array)
$notas = [7, 8, 9];
echo somaTudo(...$notas) . PHP_EOL;      // 24

// 5. RETORNO ANTECIPADO (guard clause)
function divide(float $a, float $b): ?float {
    if ($b === 0.0) return null;
    return $a / $b;
}
var_dump(divide(10, 2)); // float(5)
var_dump(divide(10, 0)); // null
```

Bloco 4: Exercício 4.1 — Calculadora Básica (4 Funções)

Exercício 4.1 — Crie 4 funções tipadas: somar, subtrair, multiplicar, dividir

```
<?php
declare(strict_types=1);

function somar(int $a, int $b): int { _____ }
function subtrair(int $a, int $b): int { _____ }
function multiplicar(int $a, int $b): int { _____ }
function dividir(int $a, int $b): ?float { _____ }

echo somar(10, 5) . PHP_EOL;           // 15
echo subtrair(10, 5) . PHP_EOL;        // 5
echo multiplicar(10, 5) . PHP_EOL;     // 50
echo dividir(10, 5) . PHP_EOL;         // 2.0
var_dump(dividir(10, 0));               // null
```

Bloco 4: Exercício 4.2 — Parâmetros Default

Exercício 4.2 — apresentar(string \$nome, int \$idade, string \$cidade = "Não informada"): string

Código inicial:

```
<?php
function apresentar(string $nome, int $idade, string
$cidade = _____): string {
    return "Nome: $nome, Idade: $idade, Cidade:
$cidade";
}

echo apresentar("Ana", 25) . PHP_EOL;
echo apresentar("Bruno", 30, "Rio de Janeiro") .
PHP_EOL;
```

Bloco 4: Exercício 4.3 — Variadic: Média

Exercício 4.3 — `media(int ...$notas): float` retorna média. Trate array vazio.

Código inicial:

```
<?php
function media(int ...$notas): float {
    // Se count($notas) === 0, retorne 0.0
    if (_____) return 0.0;
    // Use array_sum para somar
    $soma = _____;
    return $soma / count($notas);
}

echo media(7, 8, 9) . PHP_EOL;    // 8.0
echo media(10, 5) . PHP_EOL;      // 7.5
echo media() . PHP_EOL;           // 0.0
```

```
<?php
function media(int ...$notas): float {
    if (count($notas) === 0) return 0.0;
    $soma = array_sum($notas);
    return $soma / count($notas);
}

echo media(7, 8, 9) . PHP_EOL;    // 8
echo media(10, 5) . PHP_EOL;      // 7.5
echo media() . PHP_EOL;           // 0
echo media(...[6,7,8]) . PHP_EOL; // 7
```

Bloco 4: Exercício 4.4 — Higher-Order Function (callable)

Exercício 4.4 — aplicaOperacao(int \$a, int \$b, callable \$operacao): mixed

Código inicial:

```
<?php
function aplicaOperacao(int $a, int $b, callable $operacao) {
    return _____;
}

$somar = fn($x, $y) => $x + $y;
$multiplicar = fn($x, $y) => $x * $y;
$subtrair = fn($x, $y) => $x - $y;

echo aplicaOperacao(10, 5, _____) . PHP_EOL; // 15
echo aplicaOperacao(10, 5, _____) . PHP_EOL; // 50
echo aplicaOperacao(10, 5, _____) . PHP_EOL; // 5
```

Bloco 4: Exercício 4.5 — Validação e Retorno Antecipado

Exercício 4.5 — dividirSeguro(mixed \$a, mixed \$b): ?float valida tipos e zero

Código inicial:

```
<?php
function dividirSeguro(mixed $a, mixed $b): ?float {
    // Valide tipos: is_numeric() ou gettype()
    if (_____) {
        error_log("Erro: tipos inválidos");
        return _____;
    }
    // Valide divisão por zero
    if (_____) {
        error_log("Erro: divisão por zero");
        return _____;
    }
    return _____;
}

var_dump(dividirSeguro(10, 2)); // float(5)
var_dump(dividirSeguro(10, 0)); // null + log
var_dump(dividirSeguro("a", 2)); // null + log
```


Bloco 4: Desafios — Currying & Pipeline

- 4.A — Currying: `multiplicarPor(int $fator): callable` retorna função
- 4.B — Pipeline: `pipe(callable ...$fns): callable` compõe funções

Bloco 4: Desafio 4.A — Currying (Solução PHP)

```
<?php
function multiplicarPor(int $fator): callable {
    return fn(int $numero): int => $numero * $fator;
}

$dobrar = multiplicarPor(2);
$triplicar = multiplicarPor(3);
$quadruplicar = multiplicarPor(4);

echo $dobrar(5) . PHP_EOL;          // 10
echo $triplicar(5) . PHP_EOL;      // 15
echo $quadruplicar(5) . PHP_EOL;   // 20

// Versão closure clássica (PHP 7.3-)
function multiplicarPorClassic(int $fator): callable {
    return function(int $numero) use ($fator): int {
        return $numero * $fator;
    };
};
```

Bloco 4: Desafio 4.B — Pipeline/Compose (Solução PHP)

```
<?php
// Pipe: executa da esquerda para direita
function pipe(callable ...$fns): callable {
    return function($x) use ($fns) {
        foreach ($fns as $fn) {
            $x = $fn($x);
        }
        return $x;
    };
}

$dobrar = fn($x) => $x * 2;
$somar3 = fn($x) => $x + 3;
$aoQuadrado = fn($x) => $x * $x;

// h(x) = aoQuadrado(dobrar(somar3(x))) - pipe inverte ordem
$h = pipe($somar3, $dobrar, $aoQuadrado);
echo $h(5) . PHP_EOL; // aoQuadrado(dobrar(8)) = aoQuadrado(16) = 256

// Compose: executa da direita para esquerda (matemático)
function compose(callable ...$fns): callable {
    return function($x) use ($fns) {
        return array_reduce(array_reverse($fns), fn($acc, $fn) => $fn($acc), $x);
    };
}

$h2 = compose($aoQuadrado, $dobrar, $somar3);
echo $h2(5) . PHP_EOL; // 256
```

Bloco 5: Projeto Integrador — Calculadora de Estatísticas

- Objetivo: síntese de todos os 4 blocos anteriores
- Entrada: usuário define N, digita N números (validados)
- Saída: soma, média, mínimo, máximo, mediana, moda
- Requisitos: for/foreach (≥ 2 funcs), while/do...while (input), escopo correto, type hints

Bloco 5: Estrutura Completa PHP — Copie, Rode, Entenda, Explique

```
<?php
declare(strict_types=1);

// ===== FUNÇÕES PURAS (LÓGICA) =====

function soma(array $arr): int {
    $total = 0;
    for ($i = 0; $i < count($arr); $i++) {
        $total += $arr[$i];
    }
    return $total;
}

function media(array $arr): float {
    if (empty($arr)) return 0.0;
    return soma($arr) / count($arr);
}

function minimo(array $arr): int {
    $min = $arr[0];
    for ($i = 1; $i < count($arr); $i++) {
        if ($arr[$i] < $min) $min = $arr[$i];
    }
    return $min;
}

function maximo(array $arr): int {
    $max = $arr[0];
    for ($i = 1; $i < count($arr); $i++) {
        if ($arr[$i] > $max) $max = $arr[$i];
    }
    return $max;
}

function mediana(array $arr): float {
    $ordenado = $arr;
    sort($ordenado);
    $meio = (int)floor(count($ordenado) / 2);
    if (count($ordenado) % 2 === 0) {
        return ($ordenado[$meio - 1] + $ordenado[$meio]) / 2;
    }
    return (float)$ordenado[$meio];
}
```

```
function moda(array $arr): array {
    $freq = [];
    foreach ($arr as $n) {
        $freq[$n] = ($freq[$n] ?? 0) + 1;
    }
    $maxFreq = max($freq);
    $modas = [];
    foreach ($freq as $num => $f) {
        if ($f === $maxFreq) $modas[] = (int)$num;
    }
    return $modas;
}

// ===== INTERFACE (I/O) =====

function coletarNumeros(int $qtd): array {
    $nums = [];
    $i = 0;
    while ($i < $qtd) {
        $entrada = readline("Número " . ($i+1) .
            "/" . $qtd . ": ");
        $num = (int)$entrada;
        if (!is_numeric($entrada)) {
            echo "Digite um número válido!" .
                PHP_EOL;
            continue;
        }
        $nums[] = $num;
        $i++;
    }
    return $nums;
}
```

```
function main(): void {
    echo "=== CALCULADORA DE ESTATÍSTICAS ===" .
        PHP_EOL;

    $qtd = 0;
    do {
        $qtd = (int)readline("Quantos números? (mín 1):
    ");
    } while ($qtd < 1);

    $numeros = coletarNumeros($qtd);

    echo PHP_EOL . "--- RESULTADOS ---" . PHP_EOL;
    echo "Números: " . implode(", ", $numeros) .
        PHP_EOL;
    echo "Soma: " . soma($numeros) . PHP_EOL;
    echo "Média: " . number_format(media($numeros), 2)
        . PHP_EOL;
    echo "Mínimo: " . minimo($numeros) . PHP_EOL;
    echo "Máximo: " . maximo($numeros) . PHP_EOL;
    echo "Mediana: " . mediana($numeros) . PHP_EOL;
    echo "Moda: " . implode(", ", moda($numeros)) .
        PHP_EOL;
}

main();
```

Bloco 5: Checklist de Entrega — Autoavaliação

- for/foreach em pelo menos 2 funções de cálculo
- while ou do...while na validação de input (coletarNumeros / main)
- Escopo correto: variáveis locais nas funções, static se necessário
- Type hints e return types em TODAS as funções de cálculo
- Tratamento de array vazio (media retorna 0.0, min/max não quebram)
- Moda: retorna TODAS as modas em caso de empate (array)
- Extensões: desvio padrão, percentis, histograma ASCII, salvar JSON

Bloco 5: Extensões — Desvio Padrão & Percentis

```
<?php
// DESVIO PADRÃO AMOSTRAL
function desvioPadrao(array $arr): float {
    if (count($arr) < 2) return 0.0;
    $m = media($arr);
    $somaQuadrados = 0.0;
    foreach ($arr as $n) {
        $somaQuadrados += ($n - $m) ** 2;
    }
    return sqrt($somaQuadrados /
(count($arr) - 1));
}
```

```
// PERCENTIL (25%, 50%, 75%)
function percentil(array $arr, float $p): float {
    $ord = $arr;
    sort($ord);
    $idx = ($p / 100) * (count($ord) - 1);
    $base = (int)floor($idx);
    $topo = (int)ceil($idx);
    if ($base === $topo) return (float)$ord[$base];
    return $ord[$base] + ($ord[$topo] - $ord[$base]) * ($idx -
$base);
}
```

```
// HISTOGRAMA ASCII
function histograma(array $arr): void {
    $freq = [];
    foreach ($arr as $n) $freq[$n] = ($freq[$n] ?? 0) + 1;
    $maxFreq = max($freq);
    ksort($freq);
    foreach ($freq as $num => $f) {
        $barras = str_repeat("█", (int)round(($f / $maxFreq) * 20));
        echo sprintf("%3s: %s (%d)" . PHP_EOL, $num, $barras, $f);
    }
}
```

Fim do Material PHP